

Transfer Learning for Robust Classification of Hand-Drawn User Interface Sketch Elements

Anjali Barvaliya

Hochschule Furtwangen

Matrikel-Nr.: 287009

aba56562@stud.hs-furtwangen.de

Vishal Chitroda

Hochschule Furtwangen

Matrikel-Nr.: 287182

vch51367@stud.hs-furtwangen.de

Abstract—Hand-drawn user interface (UI) sketches enable rapid exploration of design ideas, yet their informal appearance makes automatic interpretation difficult. This paper evaluates transfer learning for classifying isolated UI sketch elements using 19,898 images from the UISketch dataset across 21 classes. Three learning strategies are compared under an identical protocol: a compact convolutional neural network (CNN) trained from scratch, EfficientNetB0 with a frozen ImageNet-pretrained backbone, and EfficientNetB0 with selective fine-tuning of its final 30 layers. A fixed stratified 70%/15%/15% train-validation-test split is used, and performance is assessed with accuracy, macro precision, macro recall, macro F1-score, confusion matrices, and qualitative error analysis. Fine-tuned EfficientNetB0 achieves the strongest result with 74.27% accuracy and 74.23% macro F1, compared with 49.41% accuracy and 48.25% macro F1 for the custom CNN. The findings show that pretrained features transfer effectively from natural images to sparse UI sketches and that selective fine-tuning further improves class-balanced recognition. Remaining errors are concentrated among geometrically similar components, indicating that explainability and layout context are promising directions for future work.

Index Terms—computer vision, UI sketch recognition, transfer learning, EfficientNetB0, convolutional neural networks, image classification

I. INTRODUCTION

A. Motivation

User interface design frequently begins with sketches and low-fidelity wireframes because they allow designers to explore alternatives before committing to implementation. Converting such drawings into structured labels can support design search, automatic tagging, intelligent prototyping, and later-stage sketch-to-code workflows. UISketch established a dedicated benchmark for this problem by collecting hand-drawn examples of 21 common UI element categories [1].

Sketch recognition differs from natural-image classification. Sketches contain little texture or color, contours may be incomplete, and drawing style varies considerably across users. Earlier work such as Sketch-a-Net demonstrated that sketch-specific characteristics must be considered explicitly [2], while Sketch-RNN showed that sketches can also be modeled as stroke sequences rather than only as raster images [3]. In the present study, the available samples are treated as raster images so that conventional CNNs and transfer learning can be compared under one reproducible pipeline.

Transfer learning is attractive because large-scale pretraining often produces reusable edge, contour, and shape representations [4], [5]. EfficientNet is particularly relevant because it balances network depth, width, and resolution through compound scaling [7]. EfficientNetB0 therefore offers a practical accuracy - cost trade-off for a medium-sized academic dataset.

B. Problem Statement and Contributions

The task is supervised multi-class classification. For a sketch image x , the model estimates a probability vector over $C = 21$ classes and predicts

$$\hat{y} = \arg \max_{c \in \{1, \dots, C\}} p(y = c | x). \quad (1)$$

The main difficulty is inter-class similarity. A button and a chip may both appear as rounded rectangles; a text field and a text area may share nearly identical outlines; and enabled and disabled switches may differ only through subtle stroke placement.

The contributions of this study are:

- a reproducible TensorFlow pipeline for verifying, splitting, training, and evaluating UISketch;
- a controlled comparison of learning from scratch, frozen transfer learning, and selective fine-tuning;
- a class-balanced evaluation using macro precision, macro recall, and macro F1-score;
- an error analysis grounded in confusion matrices and final-model misclassifications.

II. DATASET AND RELATED WORK

A. UISketch Dataset

The original UISketch publication reported 17,979 sketches collected across 21 categories [1]. This project contains 19,898 valid image files organized into the same 21 class folders. The implementation verifies every file programmatically rather than assuming the count from external documentation.

The available classes include alert, button, card, checkbox checked, checkbox unchecked, chip, data table, dropdown menu, floating action button, grid list, image, label, menu, radio button checked, radio button unchecked, slider, switch disabled, switch enabled, text area, text field, and tooltip.

Representative sketches from the inspected UISketch distribution.

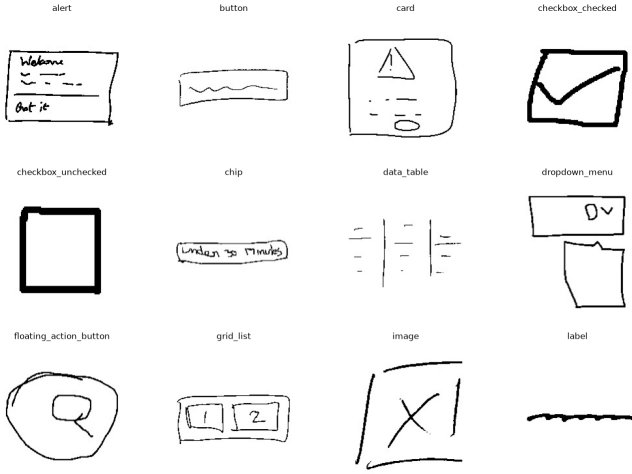


Fig. 1. Representative sketches from the inspected UISketch distribution.

The data were divided using a fixed stratified split: 13,928 images for training, 2,985 for validation, and 2,985 for testing. Stratification preserved approximately the same class proportions in all subsets.

B. Relevant Research

Sketch-a-Net showed that architectures designed around sketch statistics can outperform generic photo-oriented networks on free-hand sketch recognition [2]. Sketch-RNN explored a complementary vector-based formulation using recurrent networks [3]. These studies establish that sketches form a distinct visual domain.

For UI understanding, Rico provides a large repository of mobile interfaces with visual and structural information [17], while Enrico offers a curated benchmark for high-level UI topic classification [18]. These datasets address complete screens rather than isolated hand-drawn components, but they demonstrate the broader importance of data-driven interface analysis.

Pix2Code investigated direct generation of code from GUI screenshots [19]. Such systems require layout interpretation and sequence generation, making them considerably broader than the current study. The present work therefore focuses on reliable element classification as an earlier stage in an intelligent-interface pipeline.

III. METHODOLOGY

A. Preprocessing and Data Pipeline

Images are decoded as three-channel RGB inputs and resized to 224×224 pixels. They remain in the $[0, 255]$ range in the shared TensorFlow pipeline. The custom CNN contains an explicit rescaling layer that maps values to $[0, 1]$, whereas the TensorFlow/Keras EfficientNet implementation handles its expected preprocessing internally.

Light augmentation is applied only to training data: random rotation with factor 0.04, random zoom with factor 0.08, and

translation of up to 0.06 in each direction. These transformations represent natural variation in hand-drawn sketches without substantially changing class semantics. The pipeline uses shuffling, batching with batch size 32, and prefetching through `tf.data.AUTOTUNE`.

B. Custom CNN Baseline

The baseline contains three convolutional stages with 32, 64, and 128 filters, each followed by max pooling. Global average pooling compresses the final feature maps into a 128-dimensional representation. Two dropout layers with rates 0.4 and 0.3 reduce overfitting, and a 21-unit softmax layer produces the final class probabilities. The network contains 112,469 trainable parameters.

The custom CNN is trained for at most 25 epochs using Adam and sparse categorical cross-entropy. Early stopping monitors validation loss with patience five, while ReduceLROnPlateau reduces the learning rate by a factor of 0.3 after three stagnant epochs.

C. Frozen EfficientNetB0

EfficientNetB0 is loaded with ImageNet weights and `include_top=False`. Its convolutional backbone contains approximately 4.05 million parameters. During the frozen stage, these parameters remain fixed and only a new head is optimized. The head consists of global average pooling, dropout with rate 0.3, and a 21-unit softmax classifier, resulting in 26,901 trainable parameters.

The head is trained for at most 12 epochs. This experiment measures the value of pretrained representations without changing the feature extractor.

D. Selective Fine-Tuning

After frozen-head training, the EfficientNetB0 backbone is made trainable, but all layers except the final 30 are frozen. Batch-normalization layers remain frozen to avoid unstable moving statistics. Fine-tuning uses Adam with learning rate 10^{-5} for at most eight additional epochs. Early stopping patience is four, and the learning-rate reduction patience is two.

Only the deeper feature layers are adapted because early CNN layers encode general edges and contours, whereas later layers are more task-specific. This strategy reduces computational cost and limits catastrophic forgetting [4].

E. Evaluation Metrics

Accuracy measures overall correctness. Because class frequencies and difficulty differ, macro metrics are also reported. For class c , precision and recall are

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}. \quad (2)$$

The class-wise F1-score is

$$F1_c = \frac{2P_cR_c}{P_c + R_c}, \quad (3)$$

TABLE I
EXPERIMENTAL CONFIGURATION

Setting	Value
Input size	$224 \times 224 \times 3$
Classes	21
Split	70% / 15% / 15%
Batch size	32
Random seed	42
Optimizer	Adam [12]
Loss	Sparse categorical cross-entropy
CNN epochs	25 maximum
Frozen-head epochs	12 maximum
Fine-tuning epochs	8 maximum
Fine-tuned layers	Final 30

TABLE II
PERFORMANCE ON THE FIXED TEST SET

Model	Acc.	Macro P	Macro R	Macro F1	Stage Time
Custom CNN	49.41	50.70	48.75	48.25	170.95
EfficientNetB0 frozen	70.12	71.59	70.13	69.81	70.12
EfficientNetB0 fine-tuned	74.27	75.00	74.32	74.23	45.97

and macro F1 is

$$F1_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C F1_c. \quad (4)$$

Macro F1 gives every class equal influence and is therefore used as the primary decision metric.

IV. RESULTS AND DISCUSSION

A. Overall Performance

Table II summarizes performance on the fixed test split. The custom CNN reached 49.41% accuracy and 48.25% macro F1. Frozen EfficientNetB0 improved these values to 70.12% and 69.81%, respectively. Selective fine-tuning produced the strongest result: 74.27% accuracy, 75.00% macro precision, 74.32% macro recall, and 74.23% macro F1.

Fine-tuning improves accuracy by 24.86 percentage points over the custom CNN and by 4.15 points over the frozen model. The close agreement between accuracy and macro F1

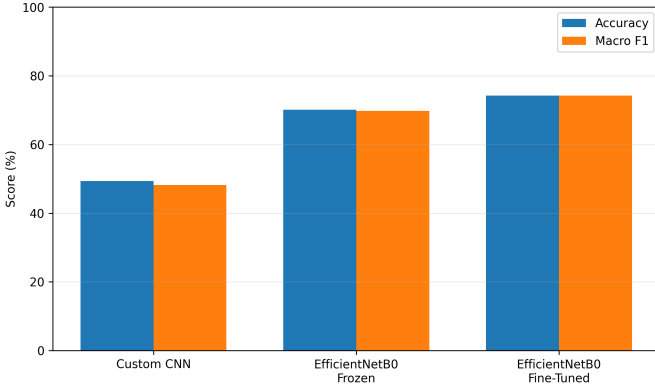


Fig. 2. Accuracy and macro F1 comparison across the three learning strategies.

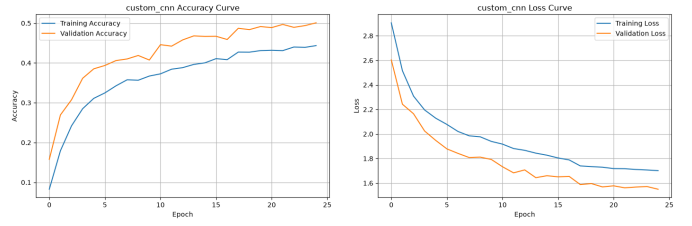


Fig. 3. Custom CNN accuracy and loss curves.

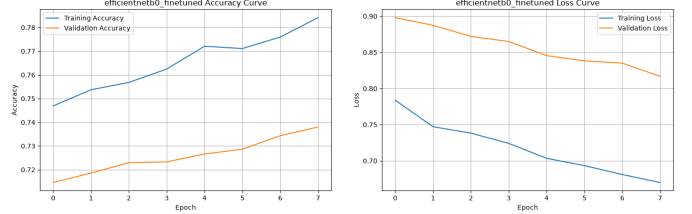


Fig. 4. Accuracy and loss during selective EfficientNetB0 fine-tuning.

indicates that the gain is not limited to a small subset of easy classes.

The reported times represent individual training stages. The 45.97 minutes shown for fine-tuning does not include the preceding frozen-head training stage and must not be interpreted as total end-to-end transfer-learning time.

B. Training Behaviour

The custom CNN curves in Fig. 3 show steady learning but a lower generalization ceiling. The model must develop both low-level and high-level features entirely from the target data. EfficientNetB0 begins with a richer representation learned from ImageNet [5], explaining the substantial performance increase in the frozen stage.

The fine-tuning curves in Fig. 4 show continued adaptation at a low learning rate. Updating only the deeper blocks allows high-level representations to become more suitable for sparse line drawings while preserving generic early features.

C. Error Analysis

The fine-tuned confusion matrix in Fig. 5 confirms that remaining errors are not uniformly distributed. Classes with distinctive structures, including floating action button, image, slider, label, grid list, and data table, are recognized more reliably. Weaker classes include alert, card, button, switch disabled, switch enabled, and text field.

The misclassified examples in Fig. 6 show that many mistakes are visually plausible. Small rounded elements may be confused with buttons or chips, rectangular containers may resemble cards or alerts, and text-oriented components may lack sufficient scale information to separate text fields from text areas.

These observations indicate that the next performance gains may require more than a stronger backbone. Full-wireframe context, relative scale, neighboring components, or explainability methods such as Grad-CAM [16] could clarify whether

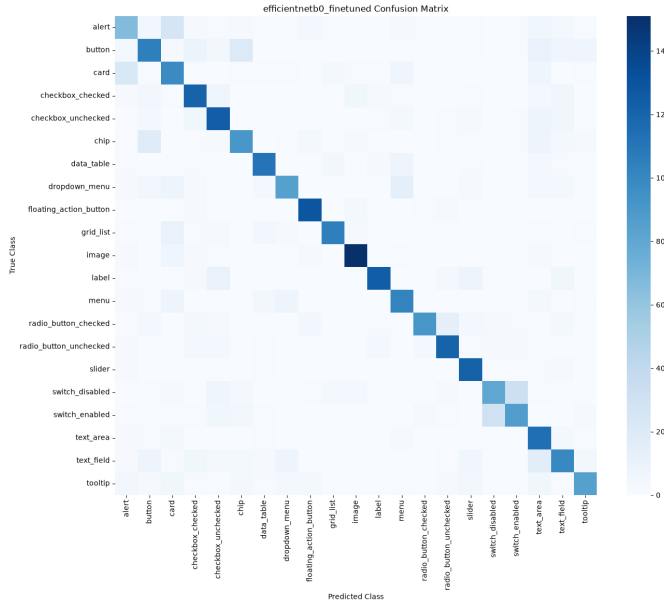


Fig. 5. Confusion matrix of the fine-tuned EfficientNetB0 model.

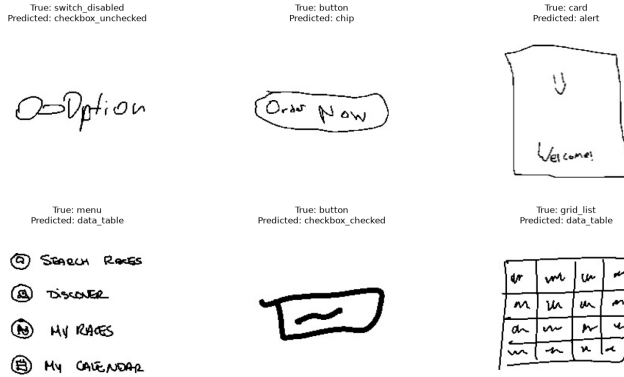


Fig. 6. Representative errors from the final fine-tuned model.

errors arise from genuine ambiguity or inappropriate model attention.

D. Limitations

The evaluation is performed on a held-out subset of the same dataset distribution; cross-dataset generalization is not measured. The model classifies one isolated element and does not infer bounding boxes, layout hierarchy, or relationships among components. Only one transfer-learning backbone is studied, so the results do not prove universal superiority over ResNet [8], MobileNet [9], or vision transformers [10]. Finally, training-time measurements depend on the local hardware, data-loading conditions, early stopping, and the number of completed epochs.

V. FUTURE WORK

Grad-CAM could be applied to correct and incorrect predictions to visualize class-relevant regions [16]. Sketch-aware

augmentation could include controlled line-thickness variation, stroke dropout, and elastic deformation, provided that label semantics are preserved. A second major direction is object detection on complete wireframes, where spatial context can help distinguish ambiguous elements. Broader UI datasets such as Rico and Enrico demonstrate the value of complete-screen structure for design understanding [17], [18]. Ultimately, element recognition could become one component in systems that translate interface imagery into structured layouts or code, as explored by Pix2Code [19].

VI. CONCLUSION

This paper presented a controlled comparison of three learning strategies for classifying 21 categories of hand-drawn UI elements. A compact custom CNN established a meaningful baseline, while frozen EfficientNetB0 demonstrated the value of ImageNet-pretrained representations. Selective fine-tuning of the final 30 layers achieved the best result with 74.27% accuracy and 74.23% macro F1.

The results confirm that transfer learning is effective for sparse UI sketches and that conservative adaptation of deeper features improves balanced recognition. At the same time, the final error analysis reveals a persistent limitation: several UI classes remain difficult to distinguish without scale and layout context. The present system should therefore be understood as a reproducible isolated-element classifier rather than a complete UI understanding or code-generation solution.

ACKNOWLEDGMENT

A large language model was used solely to assist with grammatical correction and language refinement during manuscript preparation.

REFERENCES

- [1] V. P. Sermuga Pandian, S. Suleri, and M. Jarke, "UISketch: A Large-Scale Dataset of UI Element Sketches," in *Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems*, 2021, doi: 10.1145/3411764.3445784.
- [2] Q. Yu, Y. Yang, Y.-Z. Song, T. Xiang, and T. M. Hospedales, "Sketch-a-Net that Beats Humans," in *Proc. British Machine Vision Conf.*, 2015, doi: 10.5244/C.29.7.
- [3] D. Ha and D. Eck, "A Neural Representation of Sketch Drawings," in *Proc. Int. Conf. Learning Representations*, 2018.
- [4] S. J. Pan and Q. Yang, "A Survey on Transfer Learning," *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, 2010, doi: 10.1109/TKDE.2009.191.
- [5] O. Russakovsky et al., "ImageNet Large Scale Visual Recognition Challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, 2015, doi: 10.1007/s11263-015-0816-y.
- [6] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in *Advances in Neural Information Processing Systems*, vol. 25, 2012.
- [7] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *Proc. 36th Int. Conf. Machine Learning*, pp. 6105–6114, 2019.
- [8] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, pp. 770–778, 2016.
- [9] A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv:1704.04861, 2017.
- [10] A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *Proc. Int. Conf. Learning Representations*, 2021.

- [11] M. Abadi et al., “TensorFlow: A System for Large-Scale Machine Learning,” in *Proc. 12th USENIX Symp. Operating Systems Design and Implementation*, pp. 265–283, 2016.
- [12] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *Proc. Int. Conf. Learning Representations*, 2015.
- [13] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *J. Mach. Learn. Res.*, vol. 15, pp. 1929–1958, 2014.
- [14] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *Proc. 32nd Int. Conf. Machine Learning*, pp. 448–456, 2015.
- [15] C. Shorten and T. M. Khoshgoftaar, “A Survey on Image Data Augmentation for Deep Learning,” *J. Big Data*, vol. 6, art. 60, 2019, doi: 10.1186/s40537-019-0197-0.
- [16] R. R. Selvaraju et al., “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization,” in *Proc. IEEE Int. Conf. Comput. Vis.*, pp. 618–626, 2017, doi: 10.1109/ICCV.2017.74.
- [17] B. Deka, Z. Huang, C. Franzen, J. Nichols, Y. Li, and R. Kumar, “Rico: A Mobile App Dataset for Building Data-Driven Design Applications,” in *Proc. 30th Annu. ACM Symp. User Interface Software and Technology*, pp. 845–854, 2017, doi: 10.1145/3126594.3126651.
- [18] L. A. Leiva, Y. Xue, A. Bansal, H. R. Tavakoli, T. Köroğlu, J. Du, N. R. Dayama, and A. Oulasvirta, “Enrico: A Dataset for Topic Modeling of Mobile UI Designs,” in *Proc. 22nd Int. Conf. Human-Computer Interaction with Mobile Devices and Services*, 2020, doi: 10.1145/3406324.3410710.
- [19] T. Beltramelli, “pix2code: Generating Code from a Graphical User Interface Screenshot,” in *Proc. ACM SIGCHI Symp. Engineering Interactive Computing Systems*, pp. 3–12, 2018, doi: 10.1145/3220134.3220135.
- [20] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.